

ARRANGER

BUILD | CONTROL | MANAGE

Control Command Guide DigitaLinxIP 5000 Series

Table of Contents

API Overview.....	5
License Requirements.....	5
TCP Connection	5
HTTP Requests	5
Overview of Commands.....	6
System Commands	7
reboot	7
Signal Routing Commands.....	8
join Command Summary.....	8
leave Command Summary.....	8
start / stop Command Summary.....	8
join video	9
join audio.....	10
join av	11
join usb	12
join kvm	13
join ir	14
join serial	15
join all	16
join wall.....	17
leave video.....	18
leave audio	19
leave av	20
leave usb.....	21
leave kvm	22
leave ir.....	23
leave serial.....	24
leave all	25
start	26
stop.....	27

Set Commands	28
<i>set Command Summary.....</i>	<i>28</i>
<i>set edid</i>	<i>29</i>
<i>set scaler.....</i>	<i>30</i>
<i>set video_mute</i>	<i>31</i>
<i>set video_quality</i>	<i>32</i>
<i>set frame_converter</i>	<i>33</i>
<i>set rotation</i>	<i>34</i>
<i>set audio_source</i>	<i>35</i>
<i>set volume</i>	<i>36</i>
<i>set listener</i>	<i>37</i>
<i>set var.....</i>	<i>38</i>
<i>set ui_button</i>	<i>39</i>
<i>set ui_label</i>	<i>40</i>
<i>set ui_image.....</i>	<i>41</i>
<i>set ui_page.....</i>	<i>42</i>
<i>set ui.....</i>	<i>43</i>
Get Commands.....	44
<i>get Command Summary</i>	<i>44</i>
<i>get devices.....</i>	<i>45</i>
<i>get encoder.....</i>	<i>46</i>
<i>get status.....</i>	<i>47</i>
<i>get edid.....</i>	<i>48</i>
<i>get display_status.....</i>	<i>49</i>
<i>get preferred</i>	<i>50</i>
<i>get scaler</i>	<i>51</i>
<i>get video.....</i>	<i>52</i>
<i>get video_mute</i>	<i>53</i>
<i>get video_status</i>	<i>54</i>
<i>get video_quality.....</i>	<i>55</i>
<i>get frame_converter.....</i>	<i>56</i>
<i>get rotation</i>	<i>57</i>
<i>get audio_source.....</i>	<i>58</i>
<i>get volume.....</i>	<i>59</i>
<i>get ver</i>	<i>60</i>
<i>get var</i>	<i>61</i>
<i>get ui_button</i>	<i>62</i>
<i>get ui.....</i>	<i>63</i>

send Commands	64
<i>send Command Summary</i>	64
<i>send ir</i>	65
<i>send serial</i>	66
<i>send cec</i>	68
<i>send gc</i>	69
<i>send tcp</i>	70
Preset Commands	71
<i>preset Command Summary</i>	71
<i>preset add</i>	72
<i>preset delay</i>	73
<i>preset delete</i>	74
<i>preset load</i>	75
notify Commands	76
<i>notify Command Summary</i>	76
<i>notify serial</i>	77
<i>notify network</i>	78
<i>notify display</i>	79
<i>notify source</i>	80
How to HTTP Request	81
Preset Logic	83

API Overview

This document describes everything that a developer needs to be aware of to use the Arranger command guide and develop client control applications for DigiLinuxIP 5000 series devices.

License Requirements

The Arranger controller must have a valid license key entered before use or trying to connect to the TCP control port 6980. If no valid license is active the Arranger controller will return 'Invalid License' and terminate the TCP connection. Contact Liberty AV Solutions for licensing information.

TCP Connection

Third-party controllers connect to the Arranger controller and issue commands using ASCII strings terminated with a carriage return <cr> 0x0D. This allows any TCP client to be used with the system.

The Arranger controller listens on TCP port 6980. Once a successful TCP connection is established you will receive a welcome message 'Connection Successful'.

A constant TCP connection to the Arranger controller is recommended to maintain status changes of the system from notification events.

An optional security key can be used with all TCP API commands made to port 6980.

The keyword 'key:' along with the security key are added to the API command before any parameters.

HTTP Requests

It is also possible to control the devices with HTTP GET and POST requests.

A security key must be sent with any request. Security keys are generated by 'admin' level UI users on the *Global Settings / Security Key* tab in Arranger.

GET = `http://<controllerURL>/api/command/<ARRANGER_API_COMMAND>/<KEY>`

POST = `http://<controllerURL>/api/command/{'cmd': '<ARRANGER_API_COMMAND>', 'key': '<KEY>'}`

See Section - ***How to HTTP request***

Overview of Commands

Commands are in a simple ASCII text format. For each command, the Arranger controller responds with a response which contains the return status (i.e. whether the command succeeded or not) and, if successful, the return value of the command if required.

- All commands and returns are terminated with a carriage return <cr> 0x0D
- Commands are not case sensitive
- Invalid commands will return error [Unknown]<cr>
- Missing security key will return error [security key missing]<cr>
- Invalid security key will return error [security key invalid]<cr>

Examples...

```
join video encoder1 decoder1<cr>
join video encoder1 all<cr>
join video encoder1 MyGroup<cr>
```

```
stop encoder1<cr>
start encoder1<cr>
```

```
send serial decoder1 "my data string\x0D"<cr>
```

System Commands

reboot

Used for independent routing of video signals.

Command Structure

```
reboot [key:<security_key>] <device><cr>
```

Arguments

<code><device></code>	Device name of either the encoder or the decoder
-----------------------------	--

Notes

- ***all*** is used as a destination when all devices are required to reboot.
- ***all_rx*** is used as a destination when all decoders are required to reboot.
- ***all_tx*** is used as a destination when all encoders are required to reboot.
- ***group_name*** is used as a destination when all encoders and decoders in a group are required to reboot.

Examples

Command

```
reboot Encoder1<cr>
reboot all<cr> reboot all_rx<cr>
reboot all_tx<cr> reboot MyGroup<cr>
reboot key:abc123 Encoder1<cr>
```

Response

```
reboot success<cr>

reboot error [incomplete]<cr>
reboot error [device 'Encoder1' not found]<cr>
```

Signal Routing Commands

join Command Summary

The **join** commands are used for routing signals to their desired destinations. HDMI video, audio, USB, infrared and serial signals can all be independently routed.

Commands	Description
join video	Used for independent routing of video signals
join audio	Used for independent routing of audio signals
join av	Used for independent routing of audio & video signals
join usb	Used for combined routing of USB signals
join kvm	Used for combined routing of audio, video and USB signals
join ir	Used for independent routing of IR signals
join serial	Used for independent routing of serial signals
join all	Used for combined routing of all signals
join wall	Used to join an encoder to a decoder within a video wall layout

leave Command Summary

The **leave** commands are used with a decoder to disconnect from an encoder's stream.

Commands	Description
leave video	Used to disconnect a decoder from the video stream it is receiving
leave audio	Used to disconnect a decoder from the audio stream it is receiving
leave av	Used to disconnect a decoder from both the audio and video streams it is receiving
leave usb	Used to disconnect a decoder from the USB stream it is receiving
leave kvm	Used to disconnect a decoder from the audio, video and USB streams it is receiving
leave ir	Used to disconnect a decoder from the infrared stream it is receiving
leave serial	Used to disconnect a decoder from the serial RS232 stream it is receiving
leave all	Used to disconnect a decoder from all the streams it is receiving

start / stop Command Summary

The **start / stop** commands are used to start and stop encoder signal streams on a network.

Commands	Description
start	Used to start all encoder streams
stop	Used to stop all encoder streams from being sent on the network. Joins are maintained between encoders and decoders but no data is sent from the encoder

join video

Used for independent routing of video signals.

Command Structure

```
join video [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<i><encoder></i>	Device name of the encoder
<i><decoder></i>	Device name of decoder, group or 'all'
<i><exclusive></i>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- ***all*** can replace <decoder> when the video is required to be connected to all decoders.
- The name of a ***group*** can replace <decoder> when the video is required to be connected to all decoders in a group.

Examples

Command

```
join video Encoder1 Decoder1<cr>
join video Encoder1 all<cr>
join video Encoder1 MyGroup<cr>
join video Encoder1 Decoder1 exclusive<cr>
join video Encoder1 MyGroup exclusive<cr>
join video key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join video success<cr>

join video error [incomplete]<cr>
join video error [join not permitted]<cr>
join video error [encoder 'Encoder1' not found]<cr>
join video error [decoder 'Decoder1' not found]<cr>
```

join audio

Used for independent routing of audio signals.

Command Structure

```
join audio [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
<code><decoder></code>	Device name of decoder, group or 'all'
<code><exclusive></code>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- ***all*** can replace `<decoder>` when the video is required to be connected to all decoders.
- The name of a ***group*** can replace `<decoder>` when the video is required to be connected to all decoders in a group.

Examples

Command

```
join audio Encoder1 Decoder1<cr>
join audio Encoder1 all<cr>
join audio Encoder1 MyGroup<cr>
join audio Encoder1 Decoder1 exclusive<cr>
join audio Encoder1 MyGroup exclusive<cr>
join audio key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join audio success<cr>

join audio error [incomplete]<cr>
join audio error [join not permitted]<cr>
join audio error [encoder 'Encoder1' not found]<cr>
join audio error [decoder 'Decoder1' not found]<cr>
```

join av

Used for combined routing of audio and video signals.

Command Structure

```
join av [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<encoder>	Device name of the encoder
<decoder>	Device name of decoder, group or 'all'
<exclusive>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- **all** can replace <decoder> when the video is required to be connected to all decoders.
- The name of a **group** can replace <decoder> when the video is required to be connected to all decoders in a group.

Examples

Command

```
join av Encoder1 Decoder1<cr>
join av Encoder1 all<cr>
join av Decoder1 MyGroup<cr>
join av Encoder1 MyGroup exclusive<cr>
join av Encoder1 Decoder1 exclusive<cr>
join av key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join av success<cr>

join av error [incomplete]<cr>
join av error [join not permitted]<cr>
join av error [encoder 'Encoder1' not found]<cr>
join av error [decoder 'Decoder1' not found]<cr>
```

join usb

Used for independent routing of usb signals.

Command Structure

```
join usb [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<encoder>	Device name of the encoder
<decoder>	Device name of decoder, group or 'all'
<exclusive>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- **all** can replace <decoder> when the video is required to be connected to all decoders.
- The name of a **group** can replace <decoder> when the video is required to be connected to all decoders in a group.

Examples

Command

```
join usb Encoder1 Decoder1<cr>
join usb Encoder1 all<cr>
join usb Encoder1 Mygroup<cr>
join usb Encoder1 Decoder1 exclusive<cr>
join usb Encoder1 Mygroup exclusive<cr>
join usb key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join usb success<cr>

join usb error [incomplete]<cr>
join usb error [join not permitted]<cr>
join usb error [encoder 'Encoder1' not found]<cr>
join usb error [decoder 'Decoder1' not found]<cr>
```

join kvm

Used for combined routing of audio, video and usb signals.

Command Structure

```
join kvm [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
<code><decoder></code>	Device name of decoder, group or 'all'
<code><exclusive></code>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- **all** can replace `<decoder>` when the video is required to be connected to all decoders.
- The name of a **group** can replace `<decoder>` when the video is required to be connected to all decoders in a group.

Examples

Command

```
join kvm Encoder1 Decoder1<cr>
join kvm Encoder1 all<cr>
join kvm Encoder1 MyGroup<cr>
join kvm Encoder1 MyGroup exclusive<cr>
join kvm Encoder1 Decoder1 exclusive<cr>
join kvm key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join kvm success<cr>

join kvm error [incomplete]<cr>
join kvm error [join not permitted]<cr>
join kvm error [encoder 'Encoder1' not found]<cr>
join kvm error [decoder 'Decoder1' not found]<cr>
```

join ir

Used for independent routing of ir signals

Command Structure

```
join ir [key:<security_key>] <encoder> <decoder><cr>
```

Arguments

<encoder>	Device name of the encoder
<decoder>	Device name of decoder, group or 'all'

Notes

- **all** can replace <decoder> when the video is required to be connected to all decoders.
- The name of a **group** can replace <decoder> when the video is required to be connected to all decoders in a group.

Examples

Command

```
join ir Encoder1 Decoder1<cr>
join ir Encoder1 all<cr>
join ir Encoder1 MyGroup<cr>
join ir Encoder1 Decoder1 exclusive<cr>
join ir Encoder1 MyGroup exclusive<cr>
join ir key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join ir success<cr>

join ir error [incomplete]<cr>
join ir error [join not permitted]<cr>
join ir error [source 'Encoder1' not found]<cr>
join ir error [destination 'Decoder1' not found]<cr>
```

join serial

Used for independent routing of serial signals.

Command Structure

```
join serial [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
<code><decoder></code>	Device name of decoder, group or 'all'
<code><exclusive></code>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- ***all*** can replace `<decoder>` when the video is required to be connected to all decoders.
- The name of a ***group*** can replace `<decoder>` when the video is required to be connected to all decoders in a group.

Examples

Command

```
join serial Encoder1 Decoder1<cr>
join serial Encoder1 all<cr>
join serial Encoder1 MyGroup<cr>
join serial Encoder1 MyGroup exclusive<cr>
join serial Encoder1 Decoder1 exclusive<cr>
join serial key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join serial success<cr>

join serial error [incomplete]<cr>
join serial error [join not permitted]<cr>
join serial error [encoder 'Encoder1' not found]<cr>
join serial error [decoder 'Decoder1' not found]<cr>
```

join all

Used for combined routing of ALL signals.

Command Structure

```
join all [key:<security_key>] <encoder> <decoder> [<exclusive>]<cr>
```

Arguments

<encoder>	Device name of the encoder
<decoder>	Device name of decoder, group or 'all'
<exclusive>	Keyword 'exclusive' allows for only the specified decoder to be joined with the encoder. All other decoders joined with the encoder will be removed. (optional)

Notes

- ***all*** can replace <decoder> when the video is required to be connected to all decoders.
- The name of a ***group*** can replace <decoder> when the video is required to be connected to all decoders in a group.

Examples

Command

```
join all Encoder1 Decoder1<cr>
join all Encoder1 all<cr>
join all Encoder1 MyGroup<cr>
join all Encoder1 MyGroup exclusive<cr>
join all Encoder1 Decoder1 exclusive<cr>
join all key:abc123 Encoder1 Decoder1<cr>
```

Response

```
join all success<cr>

join all error [incomplete]<cr>
join all error [join not permitted]<cr>
join all error [encoder 'Encoder1' not found]<cr>
join all error [decoder 'Decoder1' not found]<cr>
```


join wall

Used to join an encoder to a decoder within a video wall layout.

Command Structure

```
join wall [key:<security_key>] <encoder> <decoder> <wall_type>
<display_position> [size <width> <height> <fps>] [bezel <display_width>
<viewable_width> <display_height> <viewable_height>]<cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
<code><decoder></code>	Device name of decoder, group or 'all'
<code><wall_type></code>	Define the video wall configuration as columns x rows
<code><display_position></code>	Define the display position from the top left
<code>[size] (optional)</code>	Define the display resolution
<code><width></code>	Display resolution horizontal in px (used with optional size)
<code><height></code>	Display resolution vertical in px (used with optional size)
<code><fps></code>	Display frame rate (used with optional size)
<code>[bezel] (optional)</code>	Define the size of the display bezel
<code><display_width></code>	Define the overall width of the display in mm (used with optional bezel)
<code><viewable_width></code>	Define the viewable width of the display in mm (used with optional bezel)
<code><display_height></code>	Define the overall height of the display in mm (used with optional bezel)
<code><viewable_height></code>	Define the viewable height of the display in mm (used with optional bezel)

Examples

Command

```
join wall Encoder1 Decoder1 3x3 1<cr>
join wall Encoder1 Decoder1 3x3 1 size 1920 1080 60<cr>
join wall Encoder1 Decoder1 3x3 1 bezel 1000 980 800 780<cr>
join wall Encoder1 Decoder1 3x3 1 size 1920 1080 60 bezel 1000 980 800 780<cr>
```

Response

```
join wall success<cr>

join wall error [incomplete]<cr>
join wall error [join not permitted]<cr>
join wall error [encoder 'Encoder1' not found]<cr>
join wall error [decoder 'Decoder1' not found]<cr>
join wall error [invalid display_position]
```

leave video

Used to disconnect a decoder from the video stream it is receiving.

Command Structure

```
leave video [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave video Decoder1<cr>
leave video all<cr>
leave video MyGroup<cr>
leave video key:abc123 Decoder1<cr>
```

Response

```
leave video success<cr>

leave video error [incomplete]<cr>
leave video error [decoder 'Decoder1' not found]<cr>
```

leave audio

Used to disconnect a decoder from the audio stream it is receiving.

Command Structure

```
leave audio [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions

Examples

Command

```
leave audio Decoder1<cr>
leave audio all<cr>
leave audio MyGroup<cr>
leave audio key:abc123 Decoder1<cr>
```

Response

```
leave audio success<cr>

leave audio error [incomplete]<cr>
leave audio error [decoder 'Decoder1' not found]<cr>
```

leave av

Used to disconnect a decoder from both the audio and video streams it is receiving.

Command Structure

```
leave av [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave av Decoder1<cr>
leave av all<cr>
leave av MyGroup<cr>
leave av key:abc123 Decoder1<cr>
```

Response

```
leave av success<cr>

leave av error [incomplete]<cr>
leave av error [decoder 'Decoder1' not found]<cr>
```

leave usb

Used to disconnect a decoder from the usb stream it is receiving.

Command Structure

```
leave usb [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave usb Decoder1<cr>
leave usb all<cr>
leave usb MyGroup<cr>
leave usb key:abc123 Decoder1<cr>
```

Response

```
leave usb success<cr>

leave usb error [incomplete]<cr>
leave usb error [decoder 'Decoder1' not found]<cr>
```

leave kvm

Used to disconnect a decoder from the audio, video and USB streams it is receiving.

Command Structure

```
leave kvm [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave kvm Decoder1<cr>
leave kvm all<cr>
leave kvm MyGroup<cr>
leave kvm key:abc123 Decoder1<cr>
```

Response

```
leave kvm success<cr>

leave kvm error [incomplete]<cr>
leave kvm error [decoder 'Decoder1' not found]<cr>
```

leave ir

Used to disconnect a decoder from the ir stream it is receiving.

Command Structure

```
leave ir [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave ir Decoder1<cr>
leave ir all<cr>
leave ir MyGroup<cr>
leave ir key:abc123 Decoder1<cr>
```

Response

```
leave ir success<cr>

leave ir error [incomplete]<cr>
leave ir error [decoder 'Decoder1' not found]<cr>
```

leave serial

Used to disconnect a decoder from the audio, video and USB streams it is receiving.

Command Structure

```
leave serial [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave serial Decoder1<cr>
leave serial all<cr>
leave serial MyGroup<cr>
leave serial key:abc123 Decoder1<cr>
```

Response

```
leave serial success<cr>

leave serial error [incomplete]<cr>
leave serial error [decoder 'Decoder1' not found]<cr>
```


leave all

Used to disconnect a decoder from all streams it is receiving.

Command Structure

```
leave all [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
leave all Decoder1<cr>
leave all all<cr>
leave all MyGroup<cr>
leave all key:abc123 Decoder1<cr>
```

Response

```
leave all success<cr>

leave all error [incomplete]<cr>
leave all error [decoder 'Decoder1' not found]<cr>
```

start

Used to start all encoder streams.

Command Structure

```
start [key:<security_key>] <encoder>]<cr>
```

Arguments

<code><encoder></code>	Device name of the encoder or group
------------------------------	-------------------------------------

Notes

- The name of a group can replace **<encoder>** when all encoders in a group are required to stop.
- A streaming connection can be made again with the command **join** or **start**.

Examples

Command

```
start Encoder1<cr>
start MyGroup<cr>
start key:abc123 Encoder1<cr>
```

Response

```
start all success<cr>
start all error [incomplete]<cr>
start all error [encoder 'Encoder1' not found]<cr>
```

stop

Used to stop all encoder streams from being sent on the network. Joins are maintained between encoders and decoders but no data is sent from the encoder.

Command Structure

```
stop [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of decoder, group or 'all'
------------------------------	--

Notes

- ***all*** can replace <decoder> when all the decoders are required to leave video subscriptions.
- The name of a ***group*** can replace <decoder> when all decoders in a group are required to leave video subscriptions.

Examples

Command

```
stop Encoder1<cr>
stop MyGroup<cr>
stop key:abc123 Encoder1<cr>
```

Response

```
stop all success<cr>
stop all error [incomplete]<cr>
stop all error [encoder 'Encoder1' not found]<cr>
```

Set Commands

set Command Summary

The **set** commands are used to change the working conditions of an encoder or decoder.

Commands	Description
set edid	Saves EDID into encoder HDMI Input
set scaler	Changes the decoder's video output resolution
set video_mute	Disables the decoder's video output and leave a black screen
set video_quality	Manage network bandwidth by changing the video stream quality
set frame_converter	Change an encoder's video frame rate
set rotation	Rotate the video for displays that have been rotated
set audio_source	Change an encoder's audio stream from HDMI audio or analog audio
set volume	Set the volume level of the analog audio
set listener	Used to trigger presets from Global Cachè sensor notify beacons
set var	Used to store any user defined variable values
*set ui_button	Used to control a button within a <i>UI creator</i> control interface
*set ui_label	Used to control a label within a <i>UI creator</i> control interface
*set ui_image	Used to control an image within a <i>UI creator</i> control interface
*set ui	Used to enable or disable a control service within a <i>UI Creator</i>

* Requires UI Creator license purchase, see your Liberty AV Representative if you do not have a license and want to obtain this feature.

set edid

Used to save EDID into encoder HDMI Input.

Command Structure

```
set edid [key:<security_key>] <encoder> <edid><cr>
```

Arguments

<encoder>	Device name of the encoder
<edid>	String which represents the binary EDID data

Notes

- **all** can be used as a destination when all the encoders are to be set with the same EDID.
- A **group** can be used as a destination when all encoders in a group are to be set with the same EDID.
- The data argument must be a 512 character hexadecimal string which represents the EDID to be set.

Examples

Command

```
set edid Encoder1
00FFFFFFFFFFFF0010AC72A04D4439312716010380301B78EA01F5A257529F270A5054A54B00714F
8180D1C0010101010101010101010101023A801871382D40582C4500DD0C1100001E000000FF00374E30
31323239543139444D0A000000FC0044454C4C204532323131480A20000000FD00384C1E5311000A
20202020202000A1FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7F<cr>
```

```
set edid key:abc123 Encoder1 ...<cr>
```

Response

```
set edid success<cr>
```

```
set edid error [incomplete]<cr>
set edid error [invalid edid_data]<cr>
set edid error [encoder 'Encoder1' not found]<cr>
set edid error [encoder 'Encoder1' disconnected]<cr>
```

set scaler

Used to change the decoder's video output resolution.

Command Structure

```
set scaler [key:<security_key>] <decoder> <mode><cr>
```

Arguments

<i><decoder></i>	Device name of the decoder
<i><mode></i>	auto 2160p30 .. 720p25

Notes

- **Valid mode values:**
 - auto = pass-through
 - 2160p30
 - 2160p25
 - 2160p24
 - 1080p60
 - 1080p50
 - 1080p30
 - 1080p25
 - 1080p24
 - 720p60
 - 720p50
 - 720p30
 - 720p25

Examples

Command

```
set scaler Decoder1 1080p60<cr>
set scaler key:abc123 Decoder1 12<cr>
```

Response

```
set scaler success<cr>

set scaler error [incomplete]<cr>
set scaler error [invalid mode '<mode>']<cr>
set scaler error [decoder 'Decoder1' not found]<cr>
set scaler error [decoder 'Decoder1' disconnected]<cr>
```

set video_mute

Used to disable the decoder's video output and leave a black screen.

Command Structure

```
set video_mute [key:<security_key>] <decoder> <state><cr>
```

Arguments

<decoder>	Device name of the decoder
<state>	true false

Examples

Command

```
set video_mute Decoder1 true<cr>
set video_mute key:abc123 Decoder1 false<cr>
```

Response

```
set video_mute success<cr>

set video_mute error [incomplete]<cr>
set video_mute error [invalid state '<state>']<cr>
set video_mute error [decoder 'Decoder1' not found]<cr>
set video_mute error [decoder 'Decoder1' disconnected]<cr>
```

set video_quality

Used to manage network bandwidth by changing the video stream quality.

Command Structure

```
set video_quality [key:<security_key>] <encoder> <value><cr>
```

Arguments

<encoder>	Device name of the encoder
<value>	-1..5

Notes

- **Valid mode values:**
 - -1 = auto
 - 0 = minimum
 - 1
 - 2
 - 3
 - 4
 - 5 = maximum

Examples

Command

```
set video_quality Encoder1 -1<cr>
set video_quality key:abc123 Encoder1 5<cr>
```

Response

```
set video_quality success<cr>

set video_quality error [incomplete]<cr>
set video_quality error [invalid value '<value>']<cr>
set video_quality error [encoder 'Encoder1' not found]<cr>
set video_quality error [encoder 'Encoder1' disconnected]<cr>
```


set frame_converter

Used to change the encoder's video frame rate.

Command Structure

```
set frame_converter [key:<security_key>] <encoder> <value><cr>
```

Arguments

<encoder>	Device name of the encoder
<value>	0 ...59

Notes

- Value 0 is used to set the original source frame rate.
- Use the command `get frame_converter` to retrieve this setting.

Examples

Command

```
set frame_converter Encoder1 0<cr>  
set frame_converter key:abc123 Encoder1 30<cr>
```

Response

```
set frame_converter success<cr>  
  
set frame_converter error [incomplete]<cr>  
set frame_converter error [invalid value '<value>']<cr>  
set frame_converter error [encoder 'Encoder1' not found]<cr>  
set frame_converter error [encoder 'Encoder1' disconnected]<cr>
```

set rotation

Used to rotate the video for displays that have been rotated.

Command Structure

```
set rotation [key:<security_key>] <decoder> <value><cr>
```

Arguments

<i><decoder></i>	Device name of the decoder
<i><value></i>	0...7

Notes

- **Valid values:**
 - 0 = none
 - 1 = vertical mirror
 - 2 = horizontal mirror
 - 3 = 180° rotation
 - 4 = 90° clockwise rotation + horizontal mirror
 - 5 = 90° clockwise rotation
 - 6 = 270° clockwise rotation
 - 7 = 270° clockwise rotation + horizontal mirror

Examples

Command

```
set rotation Decoder1 0<cr>  
set rotation key:abc123 Decoder1 5<cr>
```

Response

```
set rotation success<cr>  
  
set rotation error [incomplete]<cr>  
set rotation error [invalid value '<value>']<cr>  
set rotation error [decoder 'Decoder1' not found]<cr>  
set rotation error [decoder 'Decoder1' disconnected]<cr>
```

set audio_source

Used to change a encoder's audio stream from HDMI audio or analog audio.

Command Structure

```
set audio_source [key:<security_key>] <encoder> <value><cr>
```

Arguments

<encoder>	Device name of the encoder
<value>	hdmi analog auto

Notes

- Use the command ***get audio_source*** to retrieve this setting.

Examples

Command

```
set audio_source Encoder1 hdmi<cr>
set audio_source Encoder1 analog<cr>
set audio_source key:abc123 Encoder1 auto<cr>
```

Response

```
set audio_source success<cr>

set audio_source error [incomplete]<cr>
set audio_source error [invalid value '<value>']<cr>
set audio_source error [encoder 'Encoder1' not found]<cr>
set audio_source error [encoder 'Encoder1' disconnected]<cr>
```

set volume

Used to to set the volume level of the analog audio.

Command Structure

```
set volume [key:<security_key>] <device> <value><cr>
```

Arguments

<i><device></i>	Device name of the encoder or decoder
<i><value></i>	0...100

Examples

Command

```
set volume Encoder1 0<cr>
set volume key:abc123 Decoder1 5<cr>
```

Response

```
set volume success<cr>

set volume error [incomplete]<cr>
set volume error [invalid value '<value>']<cr>
set volume error [device 'Encoder1' not found]<cr>
set volume error [device 'Encoder1' disconnected]<cr>
```

set listener

Used to trigger presets from Global Cachè sensor notify beacons.

Command Structure

```
set listener [key:<security_key>] <ip> <notify_port> <protocol>
<condition> <state> <device_port> <preset_name> [<delay>]<cr>
```

Arguments

<i>ip</i>	Sensor notify multicast address
<i>notify_port</i>	Sensor notify UDP/TCP port
<i>protocol</i>	UDP or TCP depending on the Global Cachè model
<i>condition</i>	Sensor notify notification of ON, OFF or BOTH
<i>state</i>	Listener state as enabled or disabled
<i>device_port</i>	Physical Global Cachè device input sensor port
<i>preset_name</i>	Name of the preset to be executed
<i>delay (optional)</i>	Optional delay time of preset execution in minutes

Notes

- UI Creator must be enabled as a feature for command to be used.

Examples

Command

```
set listener 239.255.250.250 1234 udp on true 3<cr>
set listener 239.255.250.250 1234 udp on true 3 1<cr>
set listener 239.255.250.250 1234 udp on false<cr>
```

Response

```
set listener success<cr>

set listener error [incomplete]<cr>
set listener error [invalid notify_port '<notify_port>']<cr>
set listener error [invalid device_port '<device_port>']<cr>
```

set var

Used to store any user-defined variable values.

Command Structure

```
set var [key:<security_key>] <var_name> <value> [delete]<cr>
```

Arguments

<i>var_name</i>	Name of the variable
<i>value</i>	String or boolean true false
<i>delete</i>	Keyword used to delete the variable from the system

Notes

- Value can be any string up to 256 characters.

Examples

Command

```
set var MyVar true<cr>
set var MyVar delete<cr>
set var key:abc123 MyVar false<cr>
```

Response

```
set var success<cr>

set var error [incomplete]<cr>
set var error [value max 256 charactors]<cr>
set var error [vaiable 'var_name' not found]<cr>
```

set ui_button

Used to control a button within a UI Creator control interface.

Command Structure

```
set ui_button [key:<security_key>] <ui_name> <button_name> <function>
<value><cr>
```

Arguments

<i>ui_name</i>	Name of the user interface in <i>UI Creator</i>
<i>button_name</i>	Name of the button
<i>function</i>	down state text press
<i>value</i>	true false, enabled disabled or text string depending on the function

Notes

- *UI Creator* control license must be enabled as a feature for command to be used
- function **down** uses; true | false
- function **state** uses; enabled | disabled

Examples

Command

```
set ui_button myUI myButton down true<cr>
set ui_button myUI myButton down false<cr>
set ui_button myUI myButton state enabled<cr>
set ui_button myUI myButton state disabled<cr>
set ui_button myUI myButton text MyText<cr>
set ui_button key:abc123 myUI myButton press<cr>
```

Response

```
set ui_button success<cr>

set ui_button error [incomplete]<cr>
set ui_button error [ui 'myUI' not found]<cr>
set ui_button error [button 'myButton' not found]<cr>
```

set ui_label

Used to control a label within a UI Creator control interface.

Command Structure

```
set ui_label [key:<security_key>] <ui_name> <label_name> <function>
<value><cr>
```

Arguments

<i>ui_name</i>	Name of the user interface
<i>label_name</i>	Name of the label
<i>function</i>	color visibility text
<i>value</i>	depending on the function

Notes

- *UI Creator* control license must be enabled as a feature for command to be used
- function **color** uses HEX RGB color code from 000000 to FFFFFFFF
- function **visibility** uses; true | false

Examples

Command

```
set ui_label myUI myLabel color 000000<cr>
set ui_label myUI myLabel visibility false<cr>
set ui_label myUI myLabel visibility true<cr>
set ui_label myUI myLabel text MyText<cr>
```

Response

```
set ui_label success<cr>

set ui_label error [incomplete]<cr>
set ui_label error [ui 'myUI' not found]<cr>
set ui_label error [label 'myLabel' not found]<cr>
```


set ui_image

Used to control an image within a UI Creator control interface.

Command Structure

```
set ui_image [key:<security_key>] <ui_name> <image_name> <function>
<value><cr>
```

Arguments

<i>ui_name</i>	Name of the user interface
<i>image_name</i>	Name of the image
<i>function</i>	visibility
<i>value</i>	true false

Notes

- *UI Creator* control license must be enabled as a feature for command to be used.

Examples

Command

```
set ui_image myUI myImage visibility false<cr>
set ui_image myUI myImage visibility true<cr>
```

Response

```
set ui_image success<cr>

set ui_image error [incomplete]<cr>
set ui_image error [ui 'myUI' not found]<cr>
set ui_image error [image 'myImage' not found]<cr>
```

set ui_page

The command `set ui_page` is used to change the displayed page in active user interface.

Command Structure

```
set ui_page [key:<security_key>] <ui_name> <page_name><cr>
```

Arguments

<i>ui_name</i>	Name of the user interface
<i>page_name</i>	Name of the page

Notes

- *UI Creator must be enabled as a feature for command to be used.*
- *The UI service must be enabled.*

Examples

Command

```
set ui_page myUI myPage<cr>
```

Response

```
set ui_page success<cr>
```

```
set ui_page error [incomplete]<cr>
```

```
set ui_page error [ui 'myUI' not found]<cr>
```

```
set ui_page error [page 'myPage' not found]<cr>
```

```
set ui_page error [service disabled]<cr>
```

set ui

Used to enable or disable a control service within UI Creator.

Command Structure

```
set ui [key:<security_key>] <ui_name> <service> [clients <clients>]  
[login <pin>]<cr>
```

Arguments

<i>ui_name</i>	name of the user interface
<i>service</i>	'enabled' 'disabled' 'logout'
<i>clients</i>	Optional with enabled service to set the maximum client limit from 1 to 100
<i>pin</i>	Optional with enabled service to set a fixed or random 4-digit login pin code

Notes

- *UI Creator* control license must be enabled as a feature for command to be used.
- Service **logout** is the same as disabled then enabled.

Examples

Command

```
set ui myUI disabled<cr>  
set ui myUI logout<cr>  
set ui myUI enabled<cr>  
set ui myUI enabled clients 10 login 1234<cr>  
set ui key:abc123 myUI enabled clients 10 login random<cr>
```

Response

```
set ui success<cr>  
  
set ui error [incomplete]<cr>  
set ui error [ui 'myUI' not found]<cr>  
set ui error [invalid clients '<value>']<cr>  
set ui error [invalid pin '<value>']<cr>
```

Get Commands

get Command Summary

The **get** commands are used to retrieve information from the system or encoders and decoders.

Commands	Description
get devices	Retrieves the name and MAC address of available devices
get encoder	Retrieves the encoder name subscribed to a decoder's subscription
get status	Retrieves the status of the specified device or individual encoder device stream or decoder subscription
get edid	Retrieves EDID from a decoder
get display_status	Find if a decoder has a display connected
get preferred	Retrieves the preferred resolution of a display connected to a decoder
get scaler	Retrieves the scaled video resolution of the decoder's HDMI video
get video	Retrieves the connected video information from an encoder
get video_mute	Retrieves the video mute status of the specified decoder
get video_status	Find if an encoder has a stable video source connected
get video_quality	Retrieves an encoder's video stream quality
get frame_converter	Retrieves the encoder video stream frame rate
get rotation	Retrieves the video output rotation status of the specified decoder
get audio_source	Retrieves the source of an encoder's audio stream
get volume	Retrieves the current analog audio volume level
get ver	Retrieves the current firmware version of a decoder or encoder
get var	Retrieves the value of the specified user defined variable
*get ui_button	Retrieves the current state of a user interface button
*get ui_label	Retrieves the user interface status

* Requires UI Creator license, see your Liberty AV representative if you do not have a license and want to obtain this feature.

get devices

Used to retrieve the name and MAC address of available devices.

Command Structure

```
get devices [key:<security_key>] <target><cr>
```

Arguments

<code><target></code>	<code>all all_rx all_tx</code>
-----------------------------	------------------------------------

Notes

- Return value <device_name> = name of device
- Return value <device_id> = device MAC address

Examples

Command

```
get devices all<cr>
get devices all_tx<cr>
get devices all_rx<cr>
get devices key:abc123 all<cr>
```

Response

```
get devices success '<device_name>-<device_id>','<device_name>-<device_id>'<cr>
get devices success '<device_name>-<device_id>'<cr>

get devices error [incomplete]<cr>
get devices error [invalid target '<target>']<cr>
```

get encoder

Used to retrieve the encoder name subscribed to a decoder's subscription.

Command Structure

```
get encoder [key:<security_key>] <decoder> <subscription><cr>
```

Arguments

<decoder>	Device name of the decoder
<subscription>	video audio

Examples

Command

```
get encoder Decoder1 video<cr>
get encoder Decoder1 audio<cr>
get encoder key:abc123 Decoder1 video<cr>
```

Response

```
get encoder success Encoder1 <cr>

get encoder error [incomplete]<cr>
get encoder error [invalid subscription '<subscription>']<cr>
get encoder error [no encoder connected]<cr>
get encoder error [decoder 'Decoder1' not found]<cr>
get encoder error [decoder 'Decoder1' disconnected]<cr>
```

get status

Used to retrieve the status of the specified device or individual encoder device stream or decoder subscription.

Command Structure

```
get status [key:<security_key>] <device> [<stream>]<cr>
```

Arguments

<code><device></code>	Name of the encoder or decoder
<code><stream></code>	video audio usb serial ir

Notes

- When no stream is specified the return will be as seen on the status of the encoder / decoder UI Status tab, this is a general status of the device.

Examples

Command

```
get status Encoder1<cr>
get status Encoder1 audio<cr>
get status Encoder1 video<cr>
get status Encoder1 usb<cr>
get status Encoder1 serial<cr>
get status key:abc123 Encoder1 ir<cr>
get status Decoder1<cr>
get status Decoder1 audio<cr>
get status Decoder1 video<cr>
get status Decoder1 usb<cr>
get status Decoder1 serial<cr>
get status Decoder1 ir<cr>
```

Response

```
get status success CONNECTED<cr>
get status success STOPPED<cr>
get status success TIMEOUT<cr>
get status success DISCONNECTED<cr>
get status success OUT OF RANGE<cr>

get status error [incomplete]<cr>
get status error [invalid stream '<stream>']<cr>
get status error [device 'Encoder1' not found]<cr>
get status error [device 'Encoder1' disconnected]<cr>
```

get edid

Used to retrieve a decoder's connected displays EDID table.

Command Structure

```
get edid [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of the decoder
------------------------------	----------------------------

Notes

- **all** can be used as a destination when all the encoders are to be set with the same EDID.
- A **group** can be used as a destination when all encoders in a group are to be set with the same EDID.
- The data argument must be a 512 character hexadecimal string which represents the EDID to be set.

Examples

Command

```
get edid Decoder1<cr>
get edid key:abc123 Decoder1<cr>
```

Response

```
get edid success
00FFFFFFFFFFFF0010AC72A04D4439312716010380301B78EA01F5A257529F270A5054A54B00714F
8180D1C0010101010101010101010101023A801871382D40582C4500DD0C1100001E000000FF00374E30
31323239543139444D0A000000FC0044454C4C204532323131480A20000000FD00384C1E5311000A
20202020202000A1FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF7F<cr>

get edid error [incomplete]<cr>
get edid error [decoder 'Decoder1' no EDID available]<cr>
get edid error [decoder 'Decoder1' not found]<cr>
get edid error [decoder 'Decoder1' disconnected]<cr>
```


get display_status

Used to find if a decoder has a display connected.

Command Structure

```
get display_status [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of the decoder
------------------------------	----------------------------

Notes

- Returned mode is either true or false.
- Some non-compliant displays will need to be powered on before detection is possible.

Examples

Command

```
get display_status Decoder1<cr>  
get display_status key:abc123 Decoder1<cr>
```

Response

```
get display_status success true<cr>  
get display_status success false<cr>  
  
get display_status error [incomplete]<cr>  
get display_status error [decoder 'Decoder1' not found]<cr>  
get display_status error [decoder 'Decoder1' disconnected]<cr>
```

get preferred

Used to retrieve the preferred resolution of a display connected to a decoder.

Command Structure

```
get preferred [key:<security_key>] <decoder> <option><cr>
```

Arguments

<i><device></i>	Name of the decoder
<i><option></i>	width height fps

Notes

- A display must be connected to the decoder for the EDID to be retrieved. Some non-compliant displays may need to be switched on before the EDID can be accessed.

Examples

Command

```
get preferred Decoder1 width<cr>
get preferred Decoder1 height<cr>
get preferred Decoder1 fps<cr>
get preferred key:abc123 Decoder1 width<cr>
```

Response

```
get preferred success 1920<cr>
get preferred success 1080<cr>
get preferred success 60<cr>

get preferred error [incomplete]<cr>
get preferred error [invalid option '<option>']<cr>
get preferred error [no EDID available]<cr>
get preferred error [decoder 'Decoder1' not found]<cr>
get preferred error [decoder 'Decoder1' disconnected]<cr>
```

get scaler

Used to retrieve the scaled video resolution of the decoder's HDMI video.

Command Structure

```
get scaler [key:<security_key>] <decoder> <option><cr>
```

Arguments

<device>	Name of the decoder
<option>	all width height fps

Examples

Command

```
get scaler Decoder1 all<cr>
get scaler Decoder1 width<cr>
get scaler Decoder1 height<cr>
get scaler Decoder1 fps<cr>
get scaler key:abc123 Decoder1 all<cr>
```

Response

```
get scaler success 1920 1080 60<cr>
get scaler success 1920<cr>
get scaler success 1080<cr>
get scaler success 60<cr>

get scaler error [incomplete]<cr>
get scaler error [invalid option '<option>']<cr>
get scaler error [decoder 'Decoder1' not found]<cr>
get scaler error [decoder 'Decoder1' disconnected]<cr>
```

get video

Used to retrieve the connected video information from an encoder.

Command Structure

```
get video [key:<security_key>] <encoder> <option><cr>
```

Arguments

<encoder>	Name of the encoder
<option>	all width height fps sm

Notes

- **all** returns <width> <height> <frames_per_second> <scan_mode>

Examples

Command

```
get video Encoder1 all<cr>
get video Encoder1 width<cr>
get video Encoder1 height<cr>
get video Encoder1 fps<cr>
get video Encoder1 sm<cr>
get video key:abc123 Encoder1 all<cr>
```

Response

```
get video success 1920 1080 60 PROGRESSIVE<cr>
get video success 1920<cr>
get video success 1080<cr>
get video success 60<cr>
get video success PROGRESSIVE<cr>
get video success INTERLACED <cr>

get video error [incomplete]<cr>
get video error [invalid option '<option>']<cr>
get video error [encoder 'Encoder1' not found]<cr>
get video error [encoder 'Encoder1' disconnected]<cr>
```

get video_mute

Used to retrieve the video mute status of the specified decoder.

Command Structure

```
get video_mute [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of the decoder
------------------------------	----------------------------

Notes

- Use the command ***set video_mute*** to turn it on and off or change the color of the muted display.

Examples

Command

```
get video_mute Decoder1<cr>
get video_mute key:abc123 Decoder1<cr>
```

Response

```
get video_mute success true<cr>
get video_mute success false<cr>

get video_mute error [incomplete]<cr>
get video_mute error [decoder 'Decoder1' not found]<cr>
get video_mute error [decoder 'Decoder1' disconnected]<cr>
```

get video_status

Used to find if an encoder has a stable video source connected.

Command Structure

```
get video_status [key:<security_key>] <encoder><cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
------------------------------	----------------------------

Notes

- Returned mode is either true or false.

Examples

Command

```
get video_status Encoder1<cr>
get video_status key:abc123 Encoder1<cr>
```

Response

```
get video_status success true<cr>
get video_status success false<cr>

get video_status error [incomplete]<cr>
get video_status error [encoder 'Encoder1' not found]<cr>
get video_status error [encoder 'Encoder1' disconnected]<cr>
```

get video_quality

Used to retrieve an encoder's video stream quality.

Command Structure

```
get video_quality [key:<security_key>] <encoder><cr>
```

Arguments

<code><encoder></code>	Device name of the encoder.
------------------------------	-----------------------------

Notes

- Returned value in the range of -1 to 5.
- Use the command `set video_quality` to change this setting.

Examples

Command

```
get video_quality Encoder1<cr>  
get video_quality key:abc123 Encoder1<cr>
```

Response

```
get video_status success -1<cr>  
get video_status success 5<cr>  
  
get video_quality error [incomplete]<cr>  
get video_quality error [encoder 'Encoder1' not found]<cr>  
get video_quality error [encoder 'Encoder1' disconnected]<cr>
```

get frame_converter

Used to retrieve the encoder video stream frame rate.

Command Structure

```
get frame_converter [key:<security_key>] <encoder><cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
------------------------------	----------------------------

Notes

- Use the command ***set frame_converter*** to change this setting.
- Returned values between 0 and 59.

Examples

Command

```
get frame_converter Encoder1<cr>
get frame_converter key:abc123 Encoder1<cr>
```

Response

```
get frame_converter success 0 <cr>

get frame_converter error [incomplete]<cr>
get frame_converter error [encoder 'Encoder1' not found]<cr>
get frame_converter error [encoder 'Encoder1' disconnected]<cr>
```


get rotation

Used to retrieve the video output rotation status of the specified decoder.

Command Structure

```
get rotation [key:<security_key>] <decoder><cr>
```

Arguments

<code><decoder></code>	Device name of the decoder
------------------------------	----------------------------

Notes

- Return value will be between 0 and 7.
- Use the command set rotation to change this setting.

Examples

Command

```
get rotation Decoder1<cr>  
get rotation key:abc123 Decoder1<cr>
```

Response

```
get rotation success 0<cr>  
get rotation success 7<cr>  
  
get rotation error [incomplete]<cr>  
get rotation error [decoder 'Decoder1' not found]<cr>  
get rotation error [decoder 'Decoder1' disconnected]<cr>
```

get audio_source

Used to retrieve the source of an encoder's audio stream.

Command Structure

```
get audio_source [key:<security_key>] <encoder><cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
------------------------------	----------------------------

Notes

- Returned value is either hdmi, analog or auto.
- Use the command ***set audio_source*** to change this setting.

Examples

Command

```
get audio_source Encoder1<cr>  
get audio_source key:abc123 Encoder1<cr>
```

Response

```
get audio_source success hdmi<cr>  
get audio_source success analog<cr>  
get audio_source success auto<cr>  
get audio_source error [incomplete]<cr>  
get audio_source error [encoder 'Encoder1' not found]<cr>  
get audio_source error [encoder 'Encoder1' disconnected]<cr>
```

get volume

Used to retrieve the current analog audio volume level.

Command Structure

```
get volume [key:<security_key>] <device><cr>
```

Arguments

<code><device></code>	Device name of either the encoder or decoder
-----------------------------	--

Notes

- Use the command ***set volume*** to change this setting.

Examples

Command

```
get volume Encoder1<cr>
get volume key:abc123 Decoder1<cr>
```

Response

```
get volume success 0<cr>
get volume success 100<cr>

get volume error [incomplete]<cr>
get volume error [device 'Encoder1' not found]<cr>
get volume error [device 'Encoder1' disconnected]<cr>
```

get ver

Used to retrieve the current firmware version of a decoder or encoder.

Command Structure

```
get ver [key:<security_key>] <device><cr>
```

Arguments

<code><device></code>	Device name of either the encoder or decoder
-----------------------------	--

Examples

Command

```
get ver Encoder1<cr>  
get ver key:abc123 Decoder1<cr>
```

Response

```
get ver success 1.1.2<cr>  
  
get ver error [incomplete]<cr>  
get ver error [device 'Encoder1' not found]<cr>  
get ver error [device 'Encoder1' disconnected]<cr>
```

get var

Used to retrieve the value of the specified user defined variable.

Command Structure

```
get var [key:<security_key>] <var_name><cr>
```

Arguments

<i>var_name</i>	Name of the variable
-----------------	----------------------

Examples**Command**

```
get var MyVar<cr>
```

Response

```
get var success <value><cr>
```

```
get var error [incomplete]<cr>
```

```
get var error [var '<var_name>' not found]<cr>
```

get ui_button

Retrieves the current state of a UI button in UI Creator.

Command Structure

```
get ui_button [key:<security_key>] <ui_name> <button_name>
<function><cr>
```

Arguments

<i>ui_name</i>	<i>Name of the user interface</i>
<i>button_name</i>	<i>Name of the button in the user interface</i>
<i>function</i>	<i>Down</i>

Notes

- *UI Creator control license must be enabled as a feature for command to be used.*
- Use the command ***set ui_button*** to change this setting.

Examples

Command

```
get ui_button MyUI MyButton down<cr>
get ui_button key:abc123 MyUI MyButton down<cr>
```

Response

```
get ui_button success true<cr>
get ui_button success false<cr>

get ui_button error [incomplete]<cr>
get ui_button error [User Interface '<ui_name>' not found]<cr>
get ui_button error [button '<button_name>' not found]<cr>
```

get ui

The command **get ui** is used to retrieve the user interface status.

Command Structure

```
get ui [key:<security_key>] <ui_name><cr>
```

Arguments

<code>ui_name</code>	Name of the user interface
----------------------	----------------------------

Notes

- User interface must be enabled as a feature for command to be used.
- Use the command set **ui_button** to change this setting.

Examples

Command

```
get ui MyUI<cr>
```

Response

```
get ui disabled<cr>
get ui enabled<cr>
get ui enabled timeout 240<cr>
get ui enabled clients 1<cr> *number of users 1 to 100
get ui enabled login 1234<cr> *0000 to 9999
get ui enabled timeout 240 clients 1 login 1234<cr>

get ui error [user interface 'MyUI' not found]<cr>
get ui error [service disabled]<cr>
```

send Commands

send Command Summary

The send commands are used to send either infrared or serial RS-232 data to any or all encoders or decoders from a third-party control system as well as abt TCP controllable or Global Cachè device.

Commands	Description
send ir	Sends infrared (IR) signals from a control system to encoders and decoders
send serial	Sends serial RS-232 data from a control system to encoders and decoders
send cec	Send CEC data from a control system to a decoder's connected display
send gc	Allows control over numerous Global Cache products via the same single TCP connection used for the controller
send tcp	Send a command to any TCP controllable device on the network

send serial

Used to send serial RS-232 data from a control system to encoders and decoders.

Command Structure

```
send serial [key:<security_key>] <device> "<data_string>" [<feedback>
["<feedback_string>"]]<cr>
```

Arguments

<code><device></code>	Device name of the decoder, encoder or 'all' 'all_rx' 'all_tx'
<code><data_string></code>	String of ASCII characters
<code><feedback></code>	Keywords reply, equals or contains (optional)
<code><feedback_string ></code>	String used with equals or contains to compare with the feedback string

Notes

- When either 2-way communication is required and the control system is expecting a reply from the serial equipment or unsolicited serial data is expected, then the device must be set for control mode in the RS232 serial port settings. When unsolicited serial data is received the controller will raise a 'notify serial' event.
- The ***data_string*** and ***feedback_string*** arguments are ASCII text strings when set to ASCII mode. When the device is in ASCII mode it will only be possible to send escaped \x0D carriage return and / or \x0A line feed.
 - abcdefghijklmnopqrstuvwxyz0123456789\x0D
 - \x0D\x0A
- The ***data_string*** and ***feedback_string*** arguments are ASCII text strings of bytes when set to HEX mode.
 - 00FF
 - \x00\xff
- The ***feedback*** option uses keywords reply, equals or contains to set the type of feedback.
 - To receive a string only, use reply.
 - For comparison with the specified ***feedback_string*** use equals for an exact match, or contains for a match within the string. Can only be used when ***device*** is a single encoder or decoder.
- ***feedback_string*** contains the expected device's feedback string result.

Examples

Command

```
send serial Decoder1 "my data string\x0D"<cr>
send serial Encoder1 "\x00\x01\x02\x03\x04"<cr>
send serial Decoder1 "my data string" reply<cr>
send serial Decoder1 "my data string\x0D" contains "\x0D"<cr>
send serial Decoder1 "my data string\x0D\x0A" equals "OK"<cr>
send serial key:abc123 Decoder1 "000102FF"<cr>
```

Response

```
send serial success [my return string]<cr>
send serial success [00FF0D]<cr> * Received HEX in HEX mode
send serial success [\x00\xff\x0D]<cr> * Received HEX in ASCII mode
send serial success []<cr> send serial success<cr>

send serial error [incomplete]<cr>
send serial error [invalid HEX data]<cr>
send serial error [invalid HEX feedback]<cr>
send serial error [device 'Encoder1' not found]<cr>
```

send cec

Used to send CEC data from a control system to a decoder's connected display or a source connected to an encoder.

Command Structure

```
send cec [key:<security_key>] <device> <data_hex><cr>
```

Arguments

<i><device></i>	Device name of the decoder, encoder or 'all' 'all_rx' 'all_tx'
<i><data_hex></i>	String of ascii characters representing the hexadecimal cec code

Notes

- The ***data_hex*** argument is a hexadecimal string which represent the cec code to be sent.
- The command will return with an error when using a single device if no source is connected on an encoder or no display connected on a decoder.

Examples

Command

```
send cec Decoder1 F004<cr>
send cec key:abc123 Decoder1 F004<cr>
```

Response

```
send cec success<cr>

send cec error [incomplete]<cr>
send cec error [This device does not support CEC]<cr>
send cec error [HDMI disconnected]<cr>
send cec error [decoder 'Decoder1' not found]<cr>
```

send gc

The command ***send gc*** provides a seamless integration with Global Cache products. For more information on Global Cache hardware products visit www.globalcache.com

send gc allows control over numerous Global Cache products via the same single TCP connection used for the controller.

Command Structure

```
send gc [key:<security_key>] <address> <port> <gc_api><cr>
```

Arguments

<address>	Global Cache IP address
<port>	Global Cache TCP port. Usually 4998 and for serial com1: 4999 com2: 5000
<gc_api>	Global Cache API string

Notes

- The ***gc_api*** string uses the same standard Global Cache control string format as found in the
- Use keyword "[disconnect]" in place of ***gc_api*** string to terminate the connection.
- Return values will be in the standard Global Cache format under normal conditions. An exception to this would be if a TCP connection was not possible to a device, in which case an error such as ***send gc error [device '172.30.1.111' not found]<cr>*** would be sent.

Examples

Command

```
send gc 172.30.1.111 4999 a string to send<cr> send gc 172.30.1.111 4998 setstate,1:1,1<cr>
send gc 172.30.1.111 4998 sendir,1:2,4444,34500,1,1,34,48,24,12,24,960,24,12,24...<cr>
send gc 172.30.1.111 4998 [disconnect]<cr>
send gc key:abc123 172.30.1.111 4999 a string to send<cr>
```

Response

```
a serial string<cr>          send gc error [incomplete]<cr>
state,1:1,1<cr>             send gc error [device '172.30.1.111' not found]<cr>
completeir,1:1,1<cr>        send gc error [device '172.30.1.111' timeout]<cr>
                             send gc error [invalid]<cr> ERR_<XX><cr>
```

send tcp

Issues a control command to any TCP controllable device.

Command Structure

```
send tcp [key:<security_key>] <address> <port> "<command>" [<feedback>
["<feedback_string>"]]<cr>
```

Arguments

<address>	TCP IP address
<port>	TCP port
<command>	Command string to send to TCP device
<feedback>	Keyword reply, equals or contains (optional)
<feedback_string>	Expected feedback string used with equals or contains

Notes

- The TCP device must be in the same range as the controller.
- To send HEX add \x before the HEX byte. \x0D for carriage return. \xA for line feed.
- The feedback option uses keywords reply, equals or contains to set the type of feedback.

To receive a string only, use reply.

For comparison with the specified **feedback_string** use equals for an exact match, or contains for a match within the string.

Can only be used when **device** is a single encoder or decoder.

- **feedback_string** contains the expected device's feedback string result.
- Use keyword "[disconnect]" in place of command string to terminate the connection.

Examples

Command

```
send tcp 172.30.1.111 1000 "ascii string"<cr> send tcp 172.30.1.111 1000 "an mixed string\
x0D"<cr> send tcp 172.30.1.111 1000 "\x00\x01\x02\x03"<cr>
send tcp 172.30.1.111 1000 "ascii string" contains "feedback string"<cr>
send tcp 172.30.1.111 1000 "ascii string" equals "feedback string"<cr>
send tcp 172.30.1.111 1000 "ascii string" reply<cr> send tcp 172.30.1.111 1000
[disconnect]<cr>
send tcp key:abc123 172.30.1.111 1000 "ascii string"<cr>
```

Response

```
send tcp success [<feedback>]<cr>          send tcp success [disconnected]<cr>
send tcp error [incomplete]<cr>          send tcp error [<feedback>]<cr> send tcp
error [device '172.30.1.111' not found]<cr>
send tcp error [device '172.30.1.111:1000' not connected]<cr>
```

Preset Commands

preset Command Summary

The **preset** commands are used to store and apply a series of commands available from this manual. A sequence of commands can be used to create routing tables or video wall. Refer to section **Preset logic** for logic that can be applied within a preset.

Commands	Description
*preset add	Used to create and append commands to a specified preset
*preset delay	Used within a preset to add a delay between commands
*preset delete	Used to delete the specified preset from the Arranger controller or directly from UI Creator
*preset load	Used to apply stored commands within the specified preset

* Requires UI Creator license, see your Liberty AV representative if you do not have a license and want to obtain this feature.

preset add

Used to create and append commands to a specified preset.

Command Structure

```
preset add [key:<security_key>] <preset_name> <preset_data><cr>
```

Arguments

<i>preset_name</i>	The name defined as the preset
<i>preset_data</i>	A valid control command string

Notes

- The preset is executed with the preset load command.
- Preset commands are not allowed in a preset itself, only used to create, delete and execute presets.

Examples

Command

```
preset add preset1 join all encoder1 decoder1<cr>  
preset add key:abc123 preset1 join all Encoder1 Decoder1<cr>
```

Response

```
preset add preset1 success<cr>  
  
preset add error [incomplete]<cr>
```


preset delay

Used within a preset to add a delay between commands.

Command Structure

```
preset delay [key:<security_key>] <sec>
```

Arguments

<i>sec</i>	Delay time in milliseconds up to 9999
------------	---------------------------------------

Notes

- This command can only be used within a preset.

Examples

Command

```
preset delay 1000  
preset delay key:abc123 500
```

Response

N/A

preset delete

Used to delete the specified preset from the Arranger controller or directly from UI Creator.

Command Structure

```
preset delete [key:<security_key>] <preset_name><cr>
```

Arguments

<i>preset_name</i>	The name defined as the preset
--------------------	--------------------------------

Examples

Command

```
preset delete preset1<cr>  
preset delete key:abc123 preset1<cr>
```

Response

```
preset delete preset1 success<cr>  
  
preset delete error [incomplete]<cr>  
preset delete error [preset 'preset1' not found]<cr>  
preset error [invalid mode]<cr>
```

preset load

Used to apply stored commands within the specified preset.

Command Structure

```
preset load [key:<security_key>] <preset_name> <delay><cr>
```

Arguments

<i>preset_name</i>	The name defined as the preset
<i>delay</i>	Delay in minutes before preset is applied (optional)

Notes

- The preset is created with the preset add command or directly via the UI.
- Optional “delay” is used to delay a preset for the specified amount of time in minutes. The delay is reset each time the command is used and -1 will terminate the command.

Examples

Command

```
preset load preset1<cr>
preset load preset1 30<cr>
preset load preset1 -1<cr>
preset load key:abc123 preset1<cr>
```

Response

```
preset load preset1 success<cr>

preset load error [incomplete]<cr>
preset load error [preset 'preset1' not found]<cr>
preset load preset1 error [...]<cr>
```

notify Commands

notify Command Summary

The notify messages are sent from the controller to a third-party control system connected on TCP port 6980 with updated event notifications. A notify message will be sent on the following events:

- Serial RS-232 data received from encoder or decoder
- Encoder or decoder network connectivity
- Decoder display connectivity
- Encoder source connectivity

Commands	Description
<code>notify serial</code>	Sends message when serial RS-232 data from an encoder or decoder is received.
<code>notify network</code>	Sends messages whenever an encoder or decoder is connected or disconnected from the network.
<code>notify display</code>	Sends a message whenever a display is connected or disconnected from a decoder.
<code>notify source</code>	Sends a message whenever a source is connected or disconnected from an encoder.

notify serial

Message is sent when serial RS-232 data from an encoder or decoder is received.

Command Structure

```
notify serial <device> <data_string><cr>
```

Arguments

<code><device></code>	Device name of either the encoder or decoder
<code><data_string></code>	String of ascii characters

Notes

- Before notify serial messages can be received, the device must be set to control mode.

Examples

Response

```
notify serial 'Decoder1' my data string\x0D<cr>
notify serial 'Decoder1' \x00\x01\x02\x03\xFF<cr>
notify serial 'Encoder1' 0001020304<cr>
```

notify network

Message is sent whenever an encoder or decoder is connected or disconnected from the network.

Command Structure

```
notify network <device> <state><cr>
```

Arguments

<code><device></code>	Device name of either the encoder or decoder
<code><data_string></code>	true false

Notes

- A notify network message will be sent when the controller is unable to connect or connects with an encoder or decoder. For example, a false then true message will be sent during a device power cycle or reboot.

Examples

Response

```
notify network 'Decoder1' false<cr>  
notify network 'Decoder1' true<cr>
```

notify display

Message is sent whenever a display is connected or disconnected from a decoder.

Command Structure

```
notify display <decoder> <state><cr>
```

Arguments

<i><decoder></i>	Device name of the decoder
<i><state></i>	true false

Notes

- Some non-compliant displays will require power for this event to be raised.

Examples

Response

```
notify display 'Decoder1' false<cr>  
notify display 'Decoder1' true<cr>
```

notify source

Message is sent whenever a source is connected or disconnected from an encoder

Command Structure

```
notify source <encoder> <state><cr>
```

Arguments

<code><encoder></code>	Device name of the encoder
<code><state></code>	true false

Notes

- The controller is looking for a stable valid video signal.

Examples

Response

```
notify source 'Encoder1' false<cr>  
notify source 'Encoder1' true<cr>
```


How to HTTP Request

- GET = `http://<controllerURL>/api/command/<ARRANGER_API_COMMAND>/<KEY>`
- POST = `http://<controllerURL>/api/command/{'cmd': '<ARRANGER_API_COMMAND>', 'key': '<KEY>'}`

Example 1 - POST - ajax

```
<script language='JavaScript' type='text/javascript'>

var controllerIP = '169.254.1.1'; *change this to the same IP address as the controller
var BaseURL = 'http://' + controllerIP + '/api/command/';
var MAXIMUM_WAITING_TIME = 5000; *timeout in milliseconds
var CheckStatusTimer; var key = '123xyz'; *replace this with the generated security key
var command = '123xyz'; *replace with any Arranger API command

$.ajax({
  type: 'POST',
  crossDomain: true,
  contentType: 'application/json; charset=utf-8',
  dataType: 'text',
  url: BaseURL, data: "{ 'cmd': ' + command + ', 'key': ' + key + ' }",
  timeout: MAXIMUM_WAITING_TIME,
  success: function(data, textStatus, XMLHttpRequest){
    console.log(data);
  },

  error: function (XMLHttpRequest, textStatus, errorThrown) {
    console.log('ERROR = ' + errorThrown;
  }
});

</script>
```

Example 2 - POST - xhr

```
<script language='JavaScript' type='text/javascript'>

var controllerIP = '169.254.1.1'; *change this to the same IP address as the controller
var BaseURL = 'http:// ' + controllerIP + '/api/command/';
var key = '123xyz'; *replace this with the generated security key
var command = '123xyz'; *replace with any Arranger API command
var xmlRequest = new XMLHttpRequest(); xmlRequest.open('POST', BaseURL, true);
var params = "{ 'cmd': ' + command + ' , 'key': ' + key + ' }";
var MAXIMUM_WAITING_TIME = 5000;
xmlRequest.onreadystatechange = function () {

    if (this.readyState == 4) {
        clearTimeout(xmlTimer);
        if(this.status == 200){
            console.log(this.responseText);

        }else{
            console.log('ERROR = ' + this.status + ' ' + this.statusText);
        }
    }else{
        if(this.status != 200){
            console.log('ERROR = ' + this.status + ' ' + this.statusText);
        }
    };
xmlRequest.send(params);
var xmlTimer = setTimeout(function() {
    xmlRequest.abort();
    console.log('ERROR = timeout');
}, MAXIMUM_WAITING_TIME);

</script>
```

Preset Logic

Basic **if else** logic can be applied within a preset to allow you to build some smarts into your system. All get commands can be used as an expression.

The following syntax applies:

```
if (something) {
    do_something
    ...
} else {
    do_this_instead
    ...
}
```

The following get commands will return a string value that can be used with:

- == (equal to)
- != (not equal to)

```
o get audio_source
o get devices
o get edid
o get scaler <encoder_device_name> all
o get status
o get var
o get ver
o get video <encoder_device_name> all
o get encoder
```

Example 1:

```
if (get encoder Decoder1 == Encoder1) {
    join all Encoder2 Decoder1
} else {
    join all Encoder1 Decoder2
}
```

Example 2:

```
if (get audio_source Decoder1 != hdmi) {
    set audio_source Decoder1 hdmi
}
```

The following **get** commands will return an integer value that can be used with:

- `==` (equal to)
- `!=` (not equal to)
- `<` (less than)
- `>` (greater than)

```
o get frame_converter
o get preferred
o get rotation
o get scaler <encoder_device_name> width
o get scaler <encoder_device_name> height
o get scaler <encoder_device_name> fps
o get video <encoder_device_name> width
o get video <encoder_device_name> height
o get video <encoder_device_name> fps
o get video <encoder_device_name> sm
o get video quality
o get volume
```

Example:

```
if (get rotation Encoder1 != 0) {
    set rotation Encoder1 0
} else {
    join all Encoder1 Decoder1
}
```

The following **get** commands will return a boolean value that can be used with:

- `!` (not)

```
o get display_status
o get var
o get video_mute
o get video_status
o get ui_button
```

Example 1:

```
if (get video_status Encoder1) {
    join all Encoder1 Decoder1
} else {
    if (get video_status Encoder2) {
        join all Encoder2 Decoder1
    }
}
```

Example 2:

```
if !(get video_status Encoder1) {
    join all Encoder2 Decoder1
}
```

Multiple conditional statements can be included using “&&” (and) as well as “||” (or).

```
if (get video_status Encoder1 && get video_status Encoder2) {  
    join all Encoder1 Decoder1  
    join all Encoder2 Decoder2  
}
```

```
if (get video_status Encoder1 || get video_status Encoder2) {  
    join all Encoder1 Decoder1  
    join all Encoder2 Decoder2  
}
```



LIBERTY[®]
AV SOLUTIONS

A SUBSIDIARY OF WESCO DISTRIBUTION, INC.

Phone: 719-260-0061

Fax: 719-260-0075

Toll-Free: 800-530-8998

Email: supportlibav@libav.com